

Apprentissage d'heuristiques pour solveurs CP

Programmation par contrainte

Matis Coddia & Simon Naulet

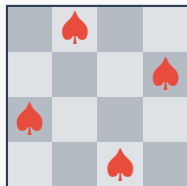
Mai 2026

Le problème : satisfaction de contraintes

Un **CSP** est défini par trois ingrédients :

- des **variables** $X_1, \dots, X_n$
- leurs **domaines** $\mathcal{D}_1, \dots, \mathcal{D}_n$
- un ensemble de **contraintes** sur ces variables

Trouver une affectation respectant toutes les contraintes.



$Q_0 = 1$

$Q_1 = 3$

$Q_2 = 0$

$Q_3 = 2$

4-Reines

4 variables Q_i , domaine $\{0, \dots, 3\}$

Contraintes :

- colonne $Q_i \neq Q_j$
- diagonale $|Q_i - Q_j| \neq |i - j|$

Cadre universel : Sudoku, planning, coloration, allocation...

Le mini-solveur : backtracking + propagation

Implémenté **from scratch** en Python : variables à domaines finis et contraintes binaires.

Backtracking

- Construction incrémentale d'une affectation
- Retour en arrière dès qu'une contrainte est violée
- **Correct et complet** par construction

Propagation (AC-3)

- Réduit les domaines par déduction logique
- Élimine les valeurs sans support
- Détecte les inconsistances en amont



La décision critique : *quelle variable choisir au branchement ?*

Phase critique : les heuristiques de branchement

L'ordre des décisions impacte **exponentiellement** la taille de l'arbre de recherche.

Heuristique	Principe	Type
Random	Choix aléatoire (baseline)	statique
First-Fail	Plus petit domaine restant	statique
Max-Degree	Plus de voisins non assignés	statique
dom/wdeg	Domaine $\div$ poids des échecs passés	dynamique
Activity	Variable la plus souvent propagée	dynamique

Intuition « fail-first » (Haralick & Elliott, 1980)

Mieux vaut échouer tôt et élaguer une grande partie de l'arbre que tard, après de nombreuses décisions à défaire.

↪ Les heuristiques dynamiques **apprennent en cours de résolution** via des compteurs internes.

L'idée : apprendre l'heuristique par ML

Constat : toutes les heuristiques sont fonctions des **mêmes signaux** (taille du domaine, degré, poids, activité, progression...).



Approche : imitation learning

- Oracle = meilleure heuristique connue (dom/wdeg)
- Le modèle apprend à *reproduire* ses choix

Pourquoi ça peut marcher

- Pondération non-linéaire des signaux
- Stratégie qui s'adapte au contexte

Transition

Du concept à l'implémentation

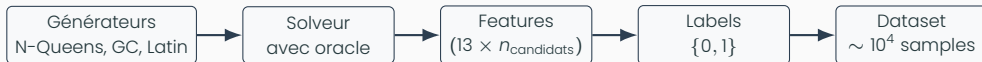
collecte → entraînement → intégration

Phase critique : collecte du dataset

Formulation : classification binaire *pointwise ranking*

À chaque nœud de décision, pour chaque variable candidate :

- extraction des **13 features** (dom, degré, wdeg, activité, etc.)
- label = 1 pour la variable choisie par l'oracle, 0 sinon



Trois familles de problèmes pour diversifier la distribution

N-Queens (8 à 14) • Graph Coloring (15 à 25 nœuds) • Latin Squares (4×4 à 6×6)

Déséquilibre fort : ~ 1 positif pour $n_{\text{candidats}}$ exemples par nœud $\rightarrow$ géré par `scale_pos_weight`.

Modèle : XGBoost et importance des features

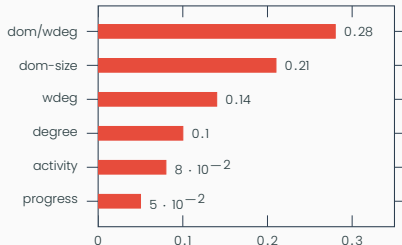
Configuration

- Gradient Boosting (200 arbres)
- Profondeur max = 6
- Early stopping (20 rounds)
- Compensation du déséquilibre

Évaluation

- Train/test stratifié (80/20)
- Accuracy + classification report

Top features apprises



Le modèle réplique ses heuristiques source

Intégration : le ML comme heuristique pluggable

Encapsulation comme une fonction d'heuristique standard du solveur.

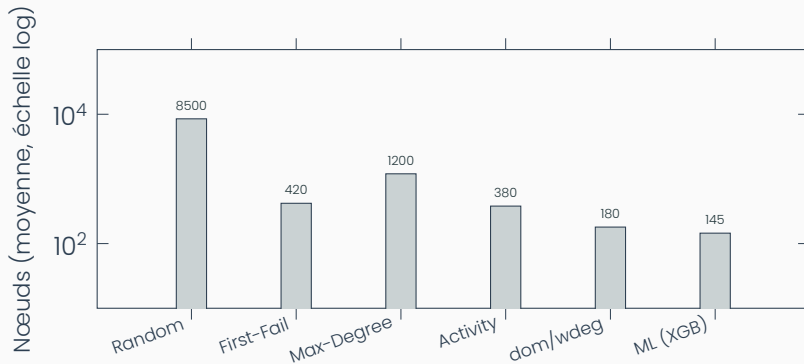
```
def ml_heuristic(csp, domains, assignment, stats):  
    unassigned = [v for v in csp.variables if v not in assignment]  
    X = np.array([  
        extract_features(...)[f] for f in FEATURE_NAMES  
        for v in unassigned  
    ])  
    probs = ml_model.predict_proba(X)[: , 1] # appel batche  
    return unassigned[np.argmax(probs)]
```

Substitution *drop-in* : aucune modification du solveur.

Conséquence

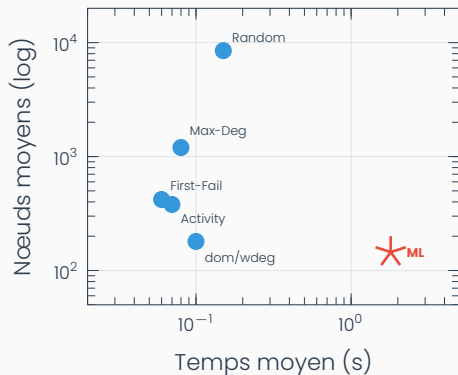
Correction et complétude préservées — le ML choisit l'ordre, jamais ne supprime des branches.

Résultats : nœuds explorés (échelle log)



- ✓ Le modèle ML **réduit le nombre de nœuds** sous l'oracle qui lui a servi à l'entraîner
- ✓ Effet de *lissage* : combinaison de plusieurs heuristiques, moins de variance
 - Mais le compte de nœuds n'est qu'une *moitié* du tableau...

Phase critique : compromis nœuds / temps



Le dilemme

Le ML explore moins de nœuds
mais chaque nœud coûte $\sim 10\times$ plus
(inférence XGBoost).

Quand le ML est-il rentable ?

- ✓ Instances **difficiles**
(beaucoup de nœuds économisés)
- ✗ Petites instances :
l'overhead domine

Conclusion et perspectives

Ce que nous avons construit

- Un mini-solveur CP complet (backtracking + AC-3)
- Cinq heuristiques classiques pour référence
- Un pipeline ML de bout en bout : collecte → entraînement → intégration

Limites identifiées

- Oracle imparfait → borne supérieure de qualité
- Overhead d'inférence non négligeable
- Généralisation hors-distribution non garantie

Perspectives

- Apprentissage par renforcement (Cappart *et al.*, 2021)
- Graph Neural Networks (Gasse *et al.*, 2019)
- Apprentissage en ligne pendant la résolution

Merci

Questions ?